Review, Comp 4915

Special Topics in MIS

- UNIX, Linux
- Kubernetes

Chapter 1: Welcome to Linux

- GNU-Linux Connection
- UNIX History
- Why Linux?
- Linux Overview
- Additional Features of Linux

History of UNIX: Linux

- 1991 - Linus Torvalds starts building Linux
  - Independent implementation of UNIX-like kernel
  - Good time frame as GNU software available
- 1994 - release 1.0 kernel
- 1996 - release 2.0 kernel
- 2003 – release 2.6 kernel
- Current stable kernel 4.14.12
  - See http://www.kernel.org/

Overview of Linux

- Multiuser, each protected from others
- Protected multitasking operating system
- Secure hierarchical file system
- “Shell” - a command interpreter and programming language
- Hundreds of useful utilities
- Choice of windows managers
- A “real” operating system

Chapter 4: Getting Started with Fedora / Red Hat Enterprise Linux

- Controlling superuser/root access
- Tour of the Fedora/RHEL Desktop
- Where to find documentation
- More about logging in
- Controlling Windows

Use man to Display System Manual

- Displays the system manual from terminal
- E.g. man ls
  - Displays the manual on the command ls
- Try man man to see the definition for the man command
- Try man bash to see the (long) short documentation on the shell
  - Command line interpreter

Command Line Syntax

- *Command [arg1] [arg2] … [argn]* ENTER
  - arguments separated by spaces (tabs too)
  - argument is something command acts upon
  - option is an argument that modifies what command does
  - many commands start options with - or --
  - sometimes can combine options (see man pages for each command)
  - ex: ls -r -x is the same as ls -rx

Command Processing

- Shell takes line and breaks it into *words* or *tokens*
  - separated by spaces
- First token taken as command name
- If command is a relative pathname, shell looks through directories in *path* for the command file
- If command file is found, shell starts a new process to run the command with the arguments
- Shell sleeps until the command is finished
- Then shell prompts for the next command

Chapter 5: The Shell

- What a shell is
- Basic features of shells
- How shell processes command
- How redirection works
- How filename expansion works
- Later unit will cover the bash shell in more detail

Functions of the Shell

- Searches for command & executes associated program
- Substitutes shell variable values
- Performs command substitution
- Performs filename generation
- Completes filenames
- Handles I/O redirection & pipelines
- Provides interpreted programming interface

Redirection

- Standard I/O files can be redirected within the shell as follows:

  `>` redirect `stdout` to file

  `>>` redirect and append `stdout` to file

  `<` redirect `stdin` from file

  `2>` redirect `stderr` to file

  `&>` redirect `stdout` and `stderr` to file

  Some examples:

```
$ cat
$ cat first last > first+last
$ cat < first+last
$ cat < last >> first
$ cat < last > first
```

Pipes

- A pipe connects the stdout of one process to the stdin of another process

  Examples:

```
$ cat food | more
$ man du | col -b | lpr
$ du -s /home/unix/* | sort -nr | head
```

- The tee utility allows us to send the output of one process to both stdout and to a file

  Example:

```
$ sort garbage | tee trash | head
```

Background Command Execution

- Append `&` to start a command in the background
  - eg: `find / -print > foo &`
- Can put the foreground job in the background by:
  - Stopping it with **`^Z`**
  - Typing **`bg`** (sends most recently stopped job to background, or **`fg`** to resume the command in the foreground)
- Can see the background jobs with **`jobs`**

```
$ jobs
[1] - Stopped      man batch
[2] - Stopped      find /usr -print
[3] + Stopped      man jobs
```

- To move a job to the foreground type `fg %<jobnum>`

```
$ fg %2
```

Filename Generation

- Following characters can be used to match existing filenames:

`*`	matches any number of characters
?	matches any single character
`[list]`	matches any character in list, can include range a-b
`[!list]`	matches any character not in list

- Can also use wildcards in pathnames
  - E.g: `$ ls /us*/[be1-3]*`

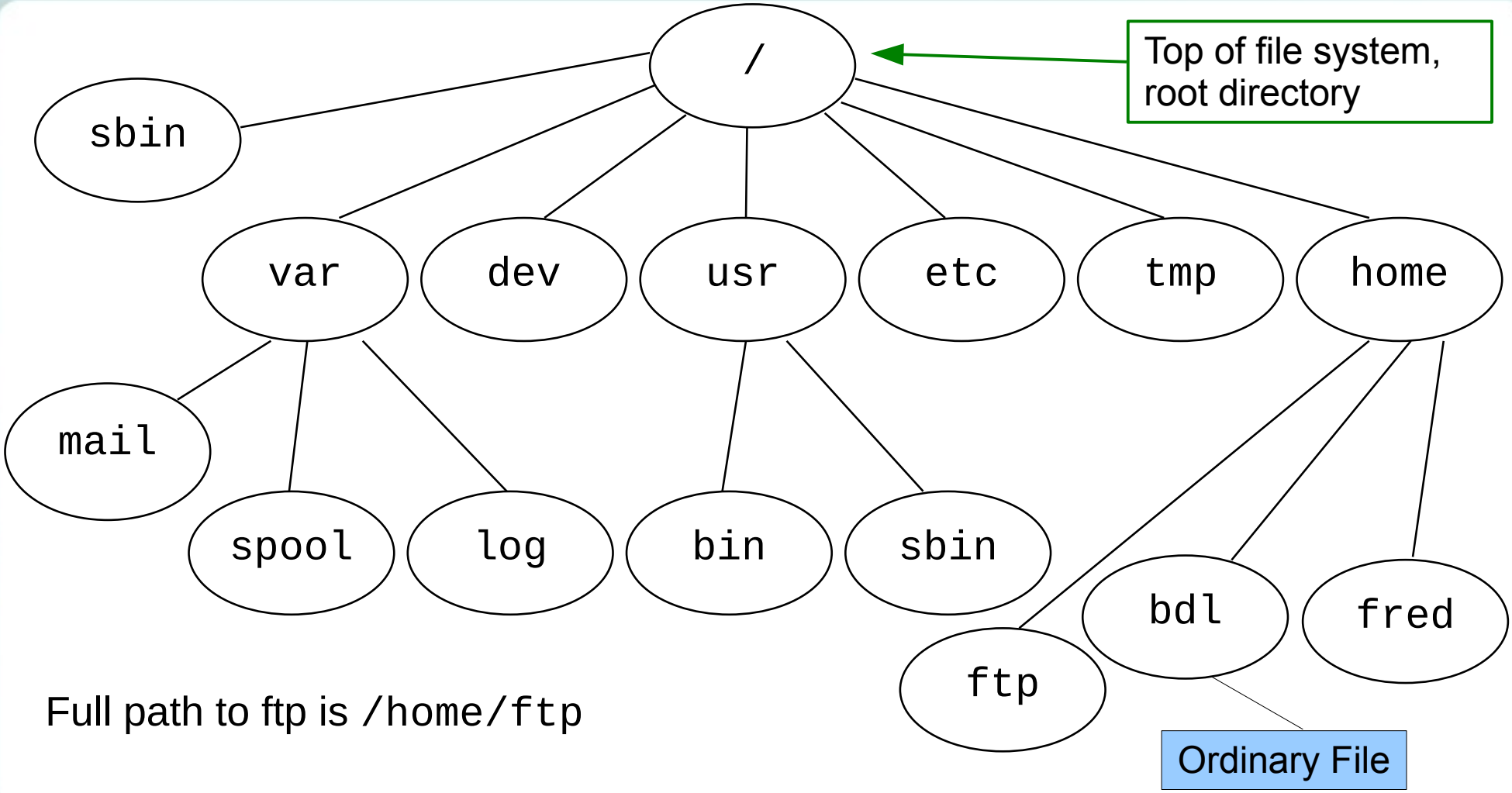
Simple Regular Expressions

- `*` matches 0 or more occurrences of the character matched to its left
  - `/ab*c/` matches ac, abc, abbc, abbbc
  - `/ab.*c/` matches abc, abxc, ab45c, ab 34.332x c
  - `/[a-zA-Z ]*/`matches a string of letters and spaces
  - `/(.*)/` matches as long a string as possible between `(` and `)`
- A regular expression matches the longest possible string, starting as far left as possible

Chapter 6: The Linux Filesystem

- Hierarchical file system
  - top of hierarchy is root directory named `/`
- The file system contains:
  - ordinary (or plain) files
  - directories
  - special files
- Provides support for linking
- Provides support for file level protection

File System



Full path to ftp is `/home/ftp`

What Precisely is a File?

- inode + data blocks
- inode is a structure containing
  - file owner
  - file group
  - access modes
  - timestamps: last accessed, modified, inode update
  - size of file (in bytes)
  - number of links to a file
  - type of file (plain, directory, etc)
  - array of pointers to data blocks

What is a Directory?

- File containing directory entries
- Directory entry: inode number + filename
- Directories always contain 2 files, `..` and `.`
- The directory you are “in” is called the “working directory” (or “current directory”)
- Home directories can be referenced with ~
  - In some shells only
- Created with `mkdir`, removed with `rmdir`

Special File Types

- 7 different types of Unix files
  - - plain files
  - d directories
  - b block special file (block I/O)
  - c character special file (char I/O)
  - l symbolic link
  - s socket
  - p named pipe

Access Permissions

- UNIX identifies the owner of a file and has permissions which control access
- File permissions control what can be done to the file
  - by the owner (user)
  - by others in the same group
  - by everyone
- Access Control Lists (ACLs) provide finer control

Access Permissions

- 3 operations on a file: read, write & execute
  - for a directory, execute means searching the directory to resolve a pathname
- Access permission depends on two things
  - type of access requested
  - who is requesting the access
- Three levels of access permissions
  - user (owner), group, other

The chmod Command

- Use chmod to change the access permissions on a file
- Syntax: `chmod <mode> filename`
- Examples:

```
chmod 777 buddy
chmod 644 kula
chmod 503 chmod
chmod u+rw multisync
chmod a+x digger
```

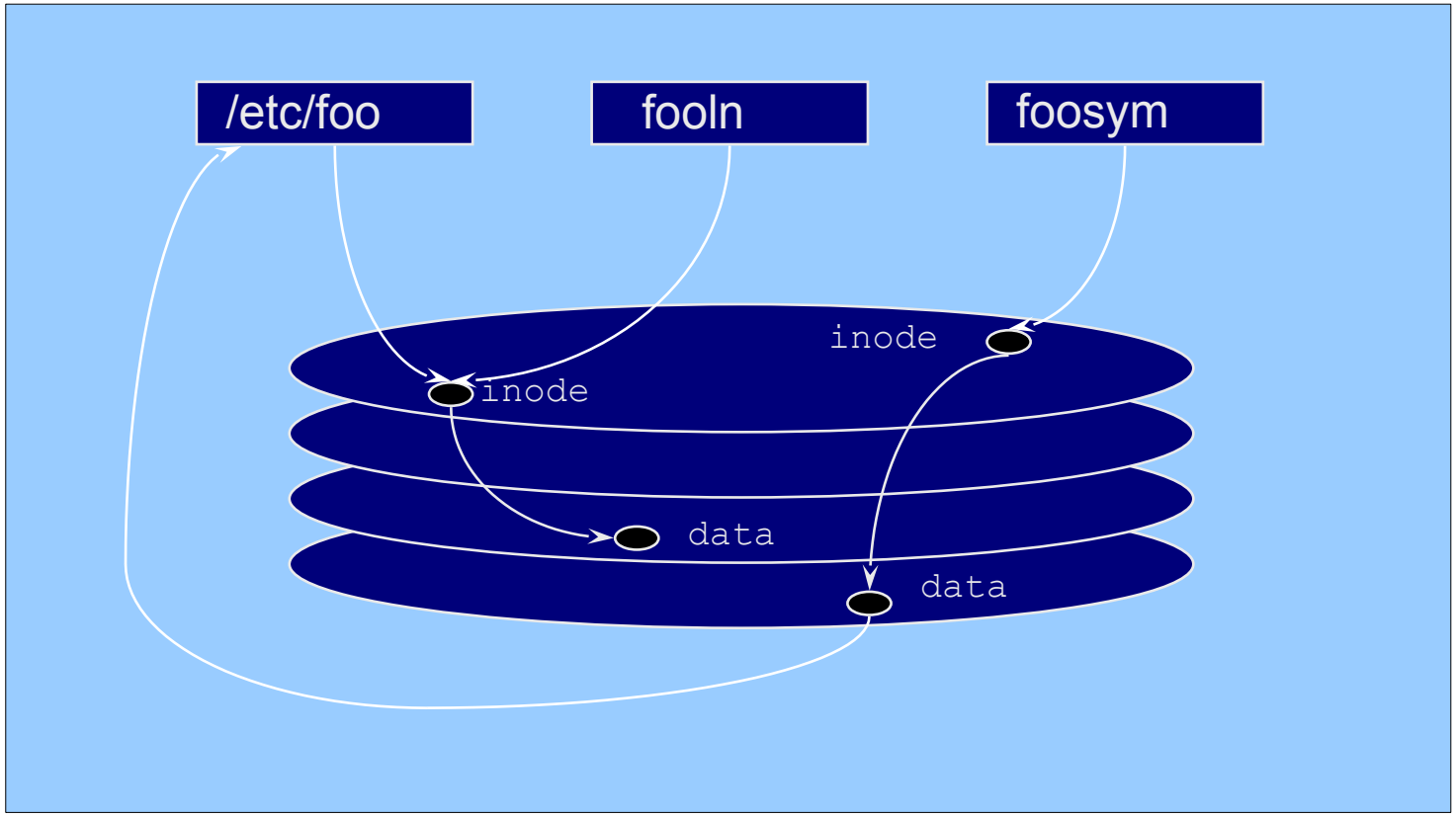
Hard Links

- Created with the `ln` command
  - eg: `ln /etc/foo fooln`
    - creates entry fooln, with same inode as /etc/foo
- A count is maintained in the inode indicating how many links there are to the file
  - Count starts at 1
- File not removed until all hard links deleted
- Hard link is considered to be a plain file
- Hard links cannot cross disk partitions

Symbolic Links

- Symbolic links are files pointing to other files
  - Have their own inode
  - Created with the `ln -s` command
- E.g.: `ln -s /etc/foo foosym`
  - Creates a new file, foosym, pointing to foo
  - foo's inode unaffected when foosym created
  - if delete /etc/foo, references to foosym generate an error (foosym points to nothing)
- Shown with type `l` files in directory listings
- Can cross disk partitions

Links (cont)



Chapter 7: The Linux Utilities

- UNIX originally had no graphical user interface
  - Had not been invented
- All of tools ran from command line
  - Tools became very powerful and efficient
- Command line utilities often faster, more powerful, more complete then GUI alternatives
- If connecting remotely to a system you may only have command line utilities available
  - Need to know them

Quoting (Escaping) Characters

- Can get rid of character's special meaning by *quoting* (or *escaping*) the character
  - \ (backslash) quotes character following it
    - Except / is a directory separator in a path even if quoted
  - Pair of single quotes (‘) quote what is between
  - Control characters can be quoted by preceding with Ctrl-V
    - Such as erase (Ctrl-H) or line kill (Ctrl-U)

Basic Utilities

- `ls`
  - Lists the files in a directory
- `cat` *`filename`*
  - Displays the contents of *filename*
- `rm` *`filename`*
  - Deletes the file named *filename* (no warning)
- `hostname`
  - Displays name of system you are logged into

File Operations

- `cp` *source-file destination-file*
  - Copies the *source-file* to the *destination-file*
  - Overwrites destination if it exists (no warning)
- `mv` *source-file destination-file*
  - Renames *source-file* to *destination-file*
  - No warning if *destination-file* already exists
- File names can have periods and are case sensitive
  - More rules when discuss file system

File Utilities

- `grep` *expression filename*
  - Displays the lines in *filename* that contain *expression*
- `head` *filename*
  - Displays first 10 lines of *filename*
- `tail` *filename*
  - Displays last 10 lines of *filename*
- `sort` *filename*
  - Displays the contents of *filename* sorted by line

File Utilities

- `uniq` *filename*
  - Displays a file, omitting consecutive duplicate lines
- `diff` *filename1 filename2*
  - Displays a list of differences between filename1 & 2
  - Useful in preparing patches
- `file` *filename*
  - Displays the type of *filename*

Compression Utilities

- `gzip` *filename*
  - compresses a file and adds a `.gz` suffix to the name
  - standard UNIX equivalent to zip
- `gunzip` *filename*
  - decompresses a file compressed with `gzip`, removes the `.gz` suffix
- `zcat` *filename*
  - decompresses a file and displays the resulting file to the terminal

Archiving

- `tar`
  - utility which handles archives of directories and files
    - see man pages for `tar` (`jar` is based on this)
  - `tar -cf` *filename list-of-files-to-archive*
    - Creates archive
  - `tar -tf` *filename*
    - Lists the contents of the archive
  - `tar -xf` *filename*
    - Recreates the directories and files from the archive

Power Commands

- These are useful in a variety of contexts
- Some will be covered in the lab assignments
  - find
  - awk
  - sed
  - perl

Networking and the Internet

Chapter 8

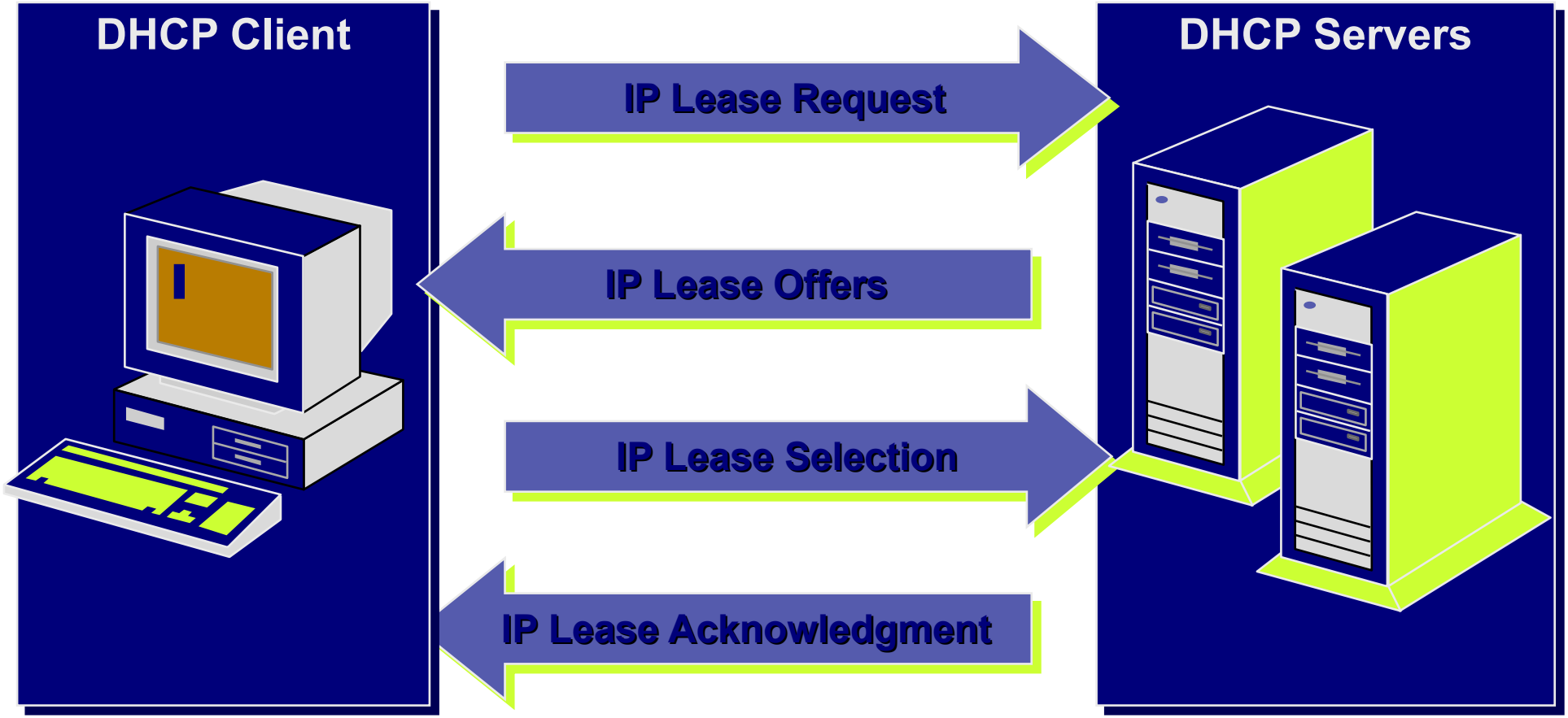
Basic internet services

What runs on top of TCP/IP

Dynamic Host Configuration Protocol (DHCP)

- DHCP Overview
- Configuring IP Addresses Using DHCP
- DHCP Requirements
- DHCP Process
- Configuring a DHCP Scope
- Other DHCP Network Settings

DHCP Process



Sample dhcpd.conf

```
subnet 142.232.66.0 netmask 255.255.255.0 {
# --- default gateway
        option routers                  142.232.66.253;
        option subnet-mask              255.255.255.0;
#       option nis-domain               "domain.org";
        option domain-name              "infosys.bcit.ca";
        option domain-name-servers      142.232.36.201;
#       option time-offset              -5;         # Eastern Standard Time
#       option ntp-servers              192.168.1.1;
        option netbios-name-servers     142.232.189.31;
        default-lease-time 21600;
        max-lease-time 43200;
# we want this computer to appear at a fixed address
        host paladin {
                hardware ethernet 00:50:DA:EA:C5:6D;
                fixed-address 142.232.66.23;
        }
}
```

Domain Name Service (DNS)

- Based on client/server model
- Distributed database system; does not rely on a single large table
- New host info sent to rest of network when needed
- When DNS server receives request for an unknown host, it passes on request to an *authoritative server*
- Authoritative server maintains accurate info about the domain being queried
- When authoritative server answers, local server caches the answer for future use

Disadvantages of Host Files

- Hard to keep up-to-date
  - Requires modifying each machine for each change
  - Administrative nightmare
- Security is easily compromised
- Easy to get duplicate host names and IP addresses
- Entries are static - doesn't deal with DHCP well
- Host file is queried before DNS
  - Errors can be hard to catch

Common DNS Record Types 1/2

- A Record
  - Specifies a host address for a name
- CNAME Record (Canonical Name)
  - Defines an alias for the hostname
- MX Record (Mail Exchange)
  - Specifies mail servers for the domain
- NS Record (Name Server)
  - Lists other name servers that contain information for a domain
  - Secondary servers for zone, primary servers for subzones

Common DNS Record Types 2/2

- PTR Record (Pointer)
  - Maps IP address to name
  - Reverses IP address and adds .in-addr.arpa to make the IP address into a name
  - 142.232.36.201 => 201.36.232.142.in-addr.arpa
- SOA Record (Start of Authority)
  - Defines the zone for which this server is authoritative
- SVR Record (Service locator)
  - Records which services are performed by which servers
  - Can list WWW servers, or Domain Controller servers

BIND

- Berkeley Internet Name Domain: an implementation of DNS
- 2 components: resolver & name server
  - resolver forms query
  - name server responds to query
- Resolver is a library of functions (“resolver code”)
- Name server is called `named`
- 3 categories of name servers
  - primary: stores domain’s info on disk file (“zone file”)
  - secondary: get zone file from primary server
  - caching-only: non-authoritative

NFS

- Machine independent network service
- Allows sharing of file systems on a network
- Implemented using RPC (Remote Procedure Call)
- Uses client/server model
  - clients request read and write operations to a server’s files
  - server reads and writes files in response to a clients request
- Any unix machine with a disk can be a server or a client or both

rwho and Other Dangers

- rwho - lists all users logged into LAN
- rwho gets info from rwhod daemon
- rwhod daemon broadcasts it’s info every few minutes
- rwhod receives broadcasts from other rwhod’s, and maintains this information

ftp

- ftp - file transfer protocol
- Syntax is `ftp remote-host`
- This is an interactive utility
- Recognizes a small set of commands (one of which is `help`)
- Used extensively on internet to transfer files to and from archive sites
  - such as rtfm.mit.edu, gatekeeper.dec.com, ftp.cdrom.com, sunsite.unc.edu
- Look for a directory called “pub”

SSH (Secure Shell)

- The R commands are quite insecure
  - Allow .hosts based authentication that can be easily spoofed
  - If l/p are exchanged, they are in cleartext, easily sniffed
- Should not be enabled on *any* machine connected directly to the internet
- SSH provides a secure replacement for rlogin, rsh, rcp

Chapter 9: bash
The Bourne Again Shell

- `bash` is a command interpreter
  - try `help` to see shell commands
  - command line syntax
  - file redirection
  - pipes and filters
  - filename generation/pathname expansion
    - special characters ? `*` `[` `]`

Specifying Shell in Scripts

- If script starts with `#!`*`program`*, shell uses *program* to interpret the rest of the file
- Usually we use the fully qualified path to a shell
  - `#!/bin/bash`
  - `#!/bin/tcsh`
- If a line starts with # not followed by `!`, it is a comment

Command Separation

- NEWLINE initiates execution of command preceding it (press ENTER)
- ; separates sequential commands on a line
- \ at the end of line escapes the NEWLINE
  - use to continue very long command on next line
- a | b causes the shell to create new processes for the commands a and b
  - output of a becomes input to b
- a& runs the command a in background

Manipulating Directory Stack

- Bash allows storing list of directories on stack
  - Makes easier to process hierarchy of directories
- `dirs` command displays stack
- `pushd` *`directory`* changes to *directory* and pushes *directory* on stack
  - pushd (no args) swaps top two directories on stack and changes directory to new top
  - pushd +*number* brings that directory to top of stack and changes to it (top of stack is +0)
- `popd` pops top of stack and changes to next top
  - `Popd +`*`number`* pops that directory, doesn't change dir

Parameters and Variables

- Shell parameters can hold values available to users and shell scripts
- Names contain letters, digits, underscores
  - Can't start with digit
- Environment variables are exported to subprocesses
  - Usually defined to be all capital letters
- User variables usually mixed or lower case
  - `myvar=abc`
  - `echo $myvar`

Positional Parameters

- The command name and parameters are stored in special variables for the use of the command
  - `$0` is the command name
  - `$1, $2`, ... are the parameters
- The `shift` builtin makes the second argument become the first, the third argument become the second, etc.
  - this can be used in a loop to process an unknown number of arguments

The `read` Builtin

- Read one line from the terminal and assign it to a variable
- Example:

```
[bdl@aeon bdl]$ cat > read1
echo -n "Go ahead: "
read firstline
echo "You entered: $firstline"
^d
[bdl@aeon bdl]$ chmod u+x read1
[bdl@aeon bdl]$ read1
Go ahead: This is a line.
You entered: This is a line.
```

Note the quotes to prevent shell from interpreting string

Special Parameters (see Table 9-6)

- $* represents all of the command-line arguments
- $@ also represents the command-line arguments
  - "$*" treats the list as a single argument
  - "$@" treats the list as a list of separate arguments
- $# contains the number of arguments
- $$ contains the process identification (PID)
- $! contains the PID of the last process run in background
- $? contains the last exit status (0 for normal exit)

Processes

- A program is an executable file residing in a disk file
- A process is an executing instance of a program
- Each process is identified by a unique nonnegative integer, its process ID (PID)
- `pstree -p` command show process tree
  - -p option includes PIDs

ps aux listing

```
[bdl@aeon bdl]$ ps aux
USER       PID %CPU %MEM   VSZ  RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.3  1104  460 ?        S    15:48   0:04 init [5]
root         2  0.0  0.0     0    0 ?        SW   15:48   0:00 [kflushd]
root         3  0.0  0.0     0    0 ?        SW   15:48   0:00 [kupdate]
root         4  0.0  0.0     0    0 ?        SW   15:48   0:00 [kpiod]
root         5  0.0  0.0     0    0 ?        SW   15:48   0:00 [kswapd]
root         6  0.0  0.0     0    0 ?        SW<  15:48   0:00 [mdrecoveryd]
bin        283  0.0  0.2  1196  288 ?        S    15:48   0:00 portmap
root       336  0.0  0.4  1152  512 ?        S    15:48   0:00 syslogd -m 0
root       347  0.0  0.5  1396  676 ?        S    15:48   0:00 klogd
daemon     363  0.0  0.2  1128  296 ?        S    15:48   0:00 /usr/sbin/atd
root       379  0.0  0.4  1304  540 ?        S    15:48   0:00 crond
root       395  0.0  0.3  1124  432 ?        S    15:48   0:00 inetd
root       411  0.0  0.3  1176  444 ?        S    15:48   0:00 lpd
root       438  0.0  0.2  1132  312 ?        S    15:48   0:00 gpm -t ps/2
bdl        537  0.0  2.9  6352 3764 ?        S    15:49   0:00 kbgndwm
bdl        538  0.0  0.2  1684  340 ?        S    15:49   0:00 sh
   /usr/bin/startkde
bdl        566  0.0  2.8  5840 3692 ?        S    15:49   0:00 krootwm
bdl        567  0.0  3.5  6600 4480 ?        S    15:49   0:01 kpanel
bdl        829  0.0  0.7  2676  964 pts/1    R    17:10   0:00 ps -aux
```

Alias

- Form is *`alias [name[=value]]`*
- If value is single-quoted, shell variables are not expanded until alias is used
- If value is double-quoted, shell variables are expanded when alias is defined
- Examples
  - `alias ls='ls -color'`
  - `alias rm='rm -i'`
  - `alias` prints the list of aliases defined
- `unalias` removes an alias

Functions

- Stores series of commands for execution later
- Stored in memory, accessed faster than shell script
- Executes in the same process that calls it
- Declared in `.bash_profile`, script, command line
- Stays until call `unset` or log out
- Precedence with commands of same name:
  - keywords such as `if` or `while` come first
  - functions
  - builtins
  - first file found with that name in the PATH

Function Syntax

```
function-name ()
{
    commands
}
```

{ can be on same line as function-name
Can also use **function** before the name

- Simple example:

```
[bdl@aeon bdl]$ whoson ()
> {
>    date
>    echo "Users Currently Logged In"
>    who
> }
[bdl@aeon bdl]$ whoson
Sun Feb 27 10:49:13 PST 2000
Users Currently Logged In
bdl      :0       Feb 27 10:08
```

Command Line Processing

- bash reads at least one line before processing
  - Entire multi-line command read, then processed
- If interactive, history expansion applied
- Alias expansion applied
- Command lines broken into tokens
- Command-line expansion applied (next)
- Command executed

Command-Line Expansion

- This is the transformation bash makes to a command line before passing it to the program called
- The shell parses the command line into tokens
- Each word is scanned for special characters
  - These lead the shell to substitute other strings for the words which appear
- Following slides describe the expansion steps in the order in which they occur
  - All expansions are fair questions

Bash Summary

- Shell is an interpreter and command language
- To execute a command, you must have execute permission
- The shell creates processes to run commands, and pipelines of commands
- Command can be in foreground or background
- Variables can be declared and expanded
- Command editing features exist
- Before running a command, the line is expanded

Chapter 10: System Administration: Core Concepts

- The job:
  - Keep one or more systems in a useful and convenient state for users
  - Can be one user (you) near system, or many systems far from your office

A Well Managed System

- Runs quickly so users are not frustrated waiting
- Has enough storage for users' reasonable needs
- Provides an environment appropriate for each user
- Is secure
- Is backed up regularly
- Has recent copies of software users need
- Is easy to administer

System Operation

- Booting the System
- Single-user mode
- Multiuser Mode
- Bringing the system down
- Crashes

Run Levels – System V

- Level 0 halts the system
- Level 1 single user
- Level 3 multiuser, terminal input
- Level 5 multiuser, XDM login
- Level 6 reboot
- Command `init` *n* changes to run level *n*

- `systemd systems more flexible`

Misc. Command Line Utilities

- `chsh` – change login shell for user
- `chgrp` - change to another group
- `dmesg` – display recent system log messages
- `e2label` – display/create label on ext2/ext3 partition
- `mkfs` – create filesystem on a device
- `ping` – send packets to remote system
- `stat` – display information about file or filesystem
- `uname` – display information about operating system

Chapter 11: Files, Directories, and Filesystems

- Important files and directories
- File Types
- Filesystems

Important Files and Directories

- Individual's initialization scripts
  - `~/.bash_profile` – login shell
  - `~/.bashrc` – noninteractive, nonlogin shell
- `In /dev/`
  - `null` – *bit bucket*, throws everything away
  - `pts` – directory mapped to pseudoterminals (`0` ...)
  - `random` and `/dev/urandom` – interfaces to kernel's random number generator
  - `zero` – source of infinite number of zeros
- Many more slides on "important files and directories"

Types of Files

- Ordinary vs. Directory Files
- Symbolic Links
- Special Files
  - FIFO files (named pipes) (p in listing)
    - Create with `mknod`
    - Allows 2 processes to exchange information
  - Sockets (s in listing)
    - Create with system call
    - Allows processes on same or different machines to exchange information
  - Block and Character Devices

Block and Character Devices

- Create with mknod
- Most are precreated for you and reside in /dev
  - E.g. `/dev/fd0`, `/dev/hda`, `/dev/mouse`
- Are used to do raw IO to/from devices
- Two kinds: Block and Character
- Characterized by major and minor device numbers
  - tty0 is device 4,0 on castor, a character device
  - hda is device 3,0 on castor, a block device

`fstab` keeps track of filesystems

- Each entry has six columns
  - Name: name of device, UUID, or volume label
  - Mount point: where in file hierarchy to mount
  - Type: type of file system, e.g. ntfs, ext3
  - Mount options: comma separated list
  - Dump: used by `dump` to schedule backups
  - Fsck: order in which `fsck` checks file systems

Chapter 15: Administration Tasks

- Configuring User and Group Accounts
- Backing up files
- System reports
- Keeping users informed
- Keeping the system secure
- Managing log files
- Disk quotas

Adding and Removing User Accounts

- To do manually, can use useradd utility as root
  - `/usr/sbin/useradd -m` *`username`*
    - `-m` creates home directory and copies standard files there
  - `passwd` *`username`*
- Manually delete account
  - `/usr/sbin/userdel -r` *`username`*
    - *`-r`* deletes user's directory
  - `usermod` to modify account
- Manually work with groups
  - `groupadd`, `groupdel`, `groupmod`

Backups

- Backups should be done on a regular basis
  - How much are you welling to lose?
- Common to do a full backup weekly and partial backups daily
  - Full backup copies all files
  - Partial backup copies files created or modified since last full backup
- Backups should be kept at different site
- Main problem - size of modern hard drives
  - May just backup user files regularly
  - System software backed up when installation changes

Chapter 18: OpenSSH: Secure Network Communication

- About OpenSSH
- OpenSSH clients
- JumpStart: using ssh and scp
- Sshd: OpenSSH server
- JumpStart: starting the sshd daemon
- Authorized keys: automatic login
- Troubleshooting
  - Tunneling/Port Forwarding

About OpenSSH

- SSH is a protocol which uses public key encryption to authorize systems, users, and set up an encrypted communication channel
  - Current version 2 incompatible with earlier ones
- Global configuration in /etc/ssh
  - ssh_config – global configuration file for SSH
  - sshd_config – configuration file for sshd
- User configuration in ~/.ssh
  - config – user overrides to ssh_config

sshd: OpenSSH Server

- If not present, install `openssh` and `openssh-server` packages
- Configure sshd to start when system is in multiuser mode
  - `/sbin/chkconfig sshd on`
- Start the first time to generate keys
  - `/sbin/service sshd start`
- Be sure firewall is open for port 22
  - Run system-configure-firewall and check SSH

Chapter 19: FTP: Transferring Files Across a Network

- FTP client
- JumpStart: Downloading Files using FTP
- Anonymous FTP
- Automatic login
- Binary versus ASCII transfer mode
- FTP server (vsftpd)
- JumpStart: Starting a vsftpd server
- vsftpd.conf: the vsftpd configuration file

File Transfer Protocol (FTP)

- Frequently used to exchange files across networks
- Covered usage in earlier module
- Two possible packages
  - `vsftpd` – “very secure ftp” (not really)
  - `wu-ftpd`
- Install with yum, rpm, etc. (as root)
  - E.g. `yum install vsftpd`
- Usually started by `inetd` or `xinetd`
- Note that FTP is an insecure protocol
  - Only use it for downloading public files

ftp>help

```
ftp> help
Commands may be abbreviated.  Commands are:

!               debug           mdir            sendport        site
$               dir             mget            put             size
account         disconnect      mkdir           pwd             status
append          exit            mls             quit            struct
ascii           form            mode            quote           system
bell            get             modtime         recv            sunique
binary          glob            mput            reget           tenex
bye             hash            newer           rstatus         tick
case            help            nmap            rhelp           trace
cd              idle            nlist           rename          type
cdup            image           ntrans          reset           user
chmod           lcd             open            restart         umask
close           ls              prompt          rmdir           verbose
cr              macdef          passive         runique         ?
delete          mdelete         proxy           send
ftp>
```

Chapter 20: Sendmail: Setting Up Mail Clients, Servers, and More

- Configuring sendmail on a client
- Configuring sendmail on a server
- How sendmail works
- Configuring sendmail
- SpamAssassin
- Webmail and mailing lists
- Setting up IMAP or POP3 server
- Authenticated relaying

Introduction to sendmail

- Using email requires three pieces of software
  - Client, called MUA (Mail User Agent)
    - mutt, KMail, Thunderbird, Outlook, Lotus Notes
    - Most can use POP, IMAP, or SMTP to receive, SMTP to send emails
  - Mail Transfer Agent (MTA) sends to destination server
    - sendmail
  - Mail Delivery Agent (MDA) puts email in recipient's mailbox
    - procmail
- Do not need to set up `sendmail` to send/receive email

How sendmail Works

- MUA passes email to sendmail
- `sendmail` separates email into headers/body and stores in `/var/spool/mqueue` directory
  - When email sent, files deleted
- Incoming mail stored in /var/spool/mail
  - Default `mbox` format
    - `maildir` format has each message in separate file
  - File per user named with user's username
  - Mail stored until collected by MUA
  - Log messages stored in /var/log/maillog

Aliases and Forwarding

- Three files can forward email: aliases, .forward, virtusertable
- `/etc/aliases` maps pseudousers to real users
  - `system:  root`
  - `abuse:  root, fred`
  - `admin:  sam, sally, aabdulla@bcit.ca`
  - `complaints:  /dev/null`
- `~/.forward` forwards a user's email
  - One destination per line
  - Destination can be local user, email address, filename, or command preceded by pipe (|)

WebMail

- Common to use web browser to read email
  - Gmail, HotMail, Yahoo! Mail
- SquirrelMail provides webmail services
  - Have to set up IMAP first and also Apache
  - Comes with plugins such as shared calendar
- To make SquirrelMail accessable from web create link
  - ln -s /usr/share/squirrelmail /var/www/html/mail
- Users get mail from http://*yourHost*/mail
- Use conf.pl script in /usr/share/squirrelmail/config to configure SquirrelMail
  - Set server's domain name at least

Chapter 21: NIS and LDAP

- NIS (Network Information Service) simplifies maintenance of common administration files on a LAN
  - Can share files such as /etc/passwd
- LDAP (Lightweight Directory Access Protocol) can hold any read-mostly structured information requiring fast lookup
  - Names, addresses, lists of network services, authentication data
  - Windows uses LDAP for its Active Directory

LDAP (Light-weight Directory Access Protocol)

- Designed to distribute information that changes infrequently (directory)
  - Fedora and RHEL use OpenLDAP
  - Optimized for fast lookup
- Data is organized into hierarchy
- LDAP clients can query and update information
- These slides give an overview of LDAP only

Directory Services

- Directories used to store information about users, accounts, and other objects
- Often have several incompatible directories
  - Mail required different account from windows account
  - Linux accounts separate from windows accounts
  - Web logins separate from OS logins
- Directory services try to combine information into one centrally managed database
  - All services and information about a network

Lightweight Directory Access Protocol (LDAP)

- A protocol for managing and querying information in a directory service over a TCP connection (port 389)
- Available on a range of platforms (Windows, Linux)
- Implemented with object-oriented database
- Information stored in *directory information tree* (DIT)
- Each object in the DIT called a *directory service entry* (DSE)
- Each DSE in a DIT has a unique name
  - *distinguished name* (DN) - entire name from root of directory
  - *relative distinguished name* (RDN) - no part or partial path to root

LDAP Naming Conventions

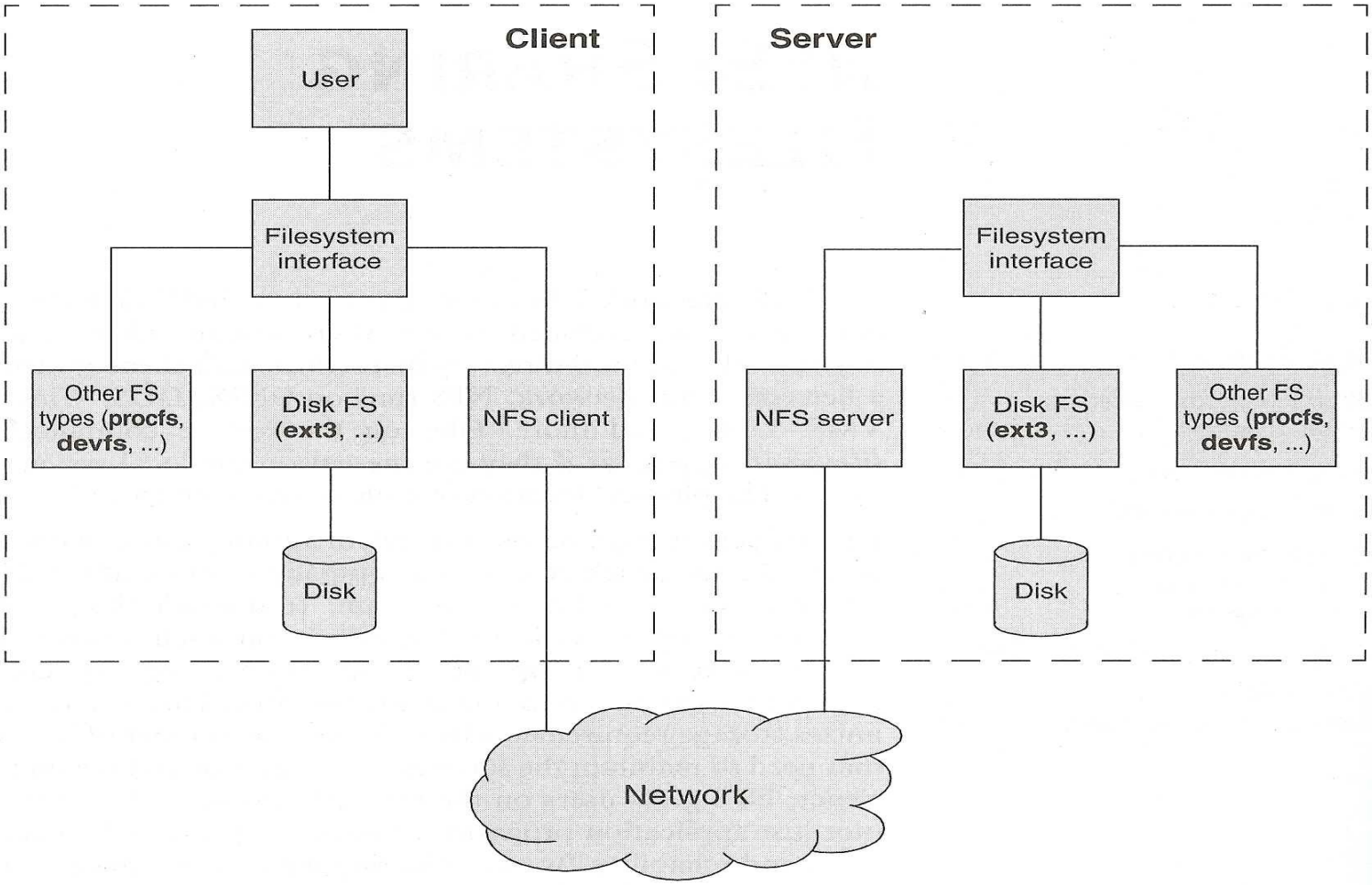
- Simple name of an object is its *common name* (CN)
- Domain objects are named with *domain component* designators (DC)
- Organizational unit objects are named with OU designators
  - Organize the database to reflect company organizations
- Example (names are not case sensitive):
  - Our domain is dc=infosys, dc=bcit, dc=ca
  - A user account for 00185586 would be

    cn=00185586,cn=users,dc=infosys,dc=bcit, dc=ca

Chapter 22: NFS: Sharing Filesystems

- Network Filesystem protocol, UNIX standard
- Files on remote server appear as if present on client
- Allows administrator to centralize copies of application programs
- Can set up diskless systems with all software/data centrally located
- Can set up dataless systems where user data centrally stored

Data Flow in NFS Setup



Chapter 23: Samba: Integrating Linux and Windows

- Samba allows UNIX-like operating systems to work with OS/2 and Windows as client or server
- Can use Samba to run a Windows Print/File server using a Linux server
  - Perhaps more efficient than using Windows server

Manually Configuring Samba Server

- Edit the /etc/samba/smb.conf file directly
- smb.conf has extensive comments and commented examples
  - Comments start with # or ;
- smb.conf file divided into sections named in [ ]
  - [globals] defines global parameters
  - [printers] defines printers
  - [homes] defines shares in homes directory
  - [*share name*] defines a share (one section each)

Samba Summary

- Allows Windows to share files and printers using SMB protocol
- User names correspond to Linux user names
  - Directly or mapped by smbusers
- Separate smbpasswd file used for passwords
- Main configuration in smb.conf
  - Simple configuration with system-config-samba
  - More complex configuration with swat
- Windows client accesses shares as usual

Chapter 24: DNS/BIND: Tracking Domain Names and Addresses

- As we saw earlier, DNS maps domain names to IP addresses
  - and vice versa, IP addresses to domain names
- Specifies protocol used to exchange information
- *Nameservers* work with clients (*resolvers*) to distribute *resource records* describing host information
- BIND is a popular open-source implementation

Resource Records

- Kept in zone files named in `named.conf` file
- Record fields
  - Name: domain name or IP address
    - if blank, inherits from previous record in same file
  - TTL: (generally optional) time to live (seconds)
  - Class: IN for internet
  - Type: record type (A, PTR, etc.)
  - Data: depends on record type

/etc/named.conf

- Establishes server as master, slave, or cache-only name server
- Specifies zones over which server has authority
- Specifies which files to read for more details

Sample named.conf

```
options {
listen-on port 53 { 127.0.0.1; 192.168.3.200; };
directory       "/var/named";
dump-file       "/var/named/data/cache_dump.db";
statistics-file "/var/named/data/named_stats.txt";
memstatistics-file
"/var/named/data/named_mem_stats.txt";
recursing-file  "/var/named/data/named.recursing";
secroots-file   "/var/named/data/named.secroots";
allow-query     { localhost; 192.168.3.0/24; };
recursion yes;
...
bindkeys-file "/etc/named.root.key";
managed-keys-directory "/var/named/dynamic";
pid-file "/run/named/named.pid";
session-keyfile "/run/named/session.key";
};

zone "." IN {
type hint;
file "named.ca";
};

include "/etc/named.rfc1912.zones";
include "/etc/named.root.key";
include "/etc/named/named.conf.local";
```

Sample named.conf.local

```
zone "infoteach.ca" {
    type master;
    file "/etc/named/zones/db.infoteach.ca"; # zone file path
};

zone "3.168.192.in-addr.arpa" {
    type master;
    file "/etc/named/zones/db.192.168.3";  # 192.168.3.0/24 subnet
};
```

Sample db.infoteach.ca

```
$TTL    604800
@       IN      SOA     services.okd4.infoteach.ca. admin.okd4.infoteach.ca. (
                1       ; Serial
           604800       ; Refresh
            86400       ; Retry
          2419200       ; Expire
           604800       ; Negative Cache TTL
)
   IN      NS      services.okd4.infoteach.ca.
services.okd4          IN      A       192.168.3.200
bootstrap.okd4         IN      A       192.168.3.230
control-plane-1.okd4   IN      A       192.168.3.201
control-plane-2.okd4   IN      A       192.168.3.202
control-plane-3.okd4   IN      A       192.168.3.203
compute-1.okd4         IN      A       192.168.3.204
compute-2.okd4         IN      A       192.168.3.205
...
; OpenShift internal cluster IPs - A records
api.okd4    IN    A    192.168.3.200
api-int.okd4    IN    A    192.168.3.200
*.apps.okd4    IN    A    192.168.3.200

okd4   IN CNAME services.okd4
```

Chapter 25: iptables: Setting Up a Firewall

- The iptables utility builds and manipulates network packet filtering rules in Linux kernel
  - Can protect system from malicious users
  - Also set up NAT (Network Address Translation)
    - allow multiple users to share Internet connection
  - Provides connection tracking to handle packets based on state of their connection
- Text covers fundamentals only
- First a discussion of security at a higher level

What are the Risks

- Theft
  - steal information for personal gain
- Sabotage
  - destroy files, servers
  - alter files
- Denial of Service
  - virus attacks
  - slow down systems

Network Security

- Problems associated with the network can come from the inside or outside
  - Can be malicious, accidental, or curiosity
- Should set up log files to catch suspicious activity and read them daily
- Port security is critical
  - Know all the ports and services on your system
  - Look for activity on unauthorized ports

Unauthorized Access

- Most TCP/IP services are accessible from any system unless restricted
  - NFS, NIS, SAMBA
  - HTTP, FTP, SSH
  - RLOGIN, RSH, TELNET
- Think who should access these and restrict access accordingly
  - Disable risky services and restrict allowed clients for the rest

Firewall Functionality

- Firewall software are configured with rules
  - What happens with each network packet
  - Packets can pass, be blocked, or modified
  - Rules can handle both incoming and outgoing packets
- Network designed with firewalls, servers and client computers
  - To achieve reasonable security and access

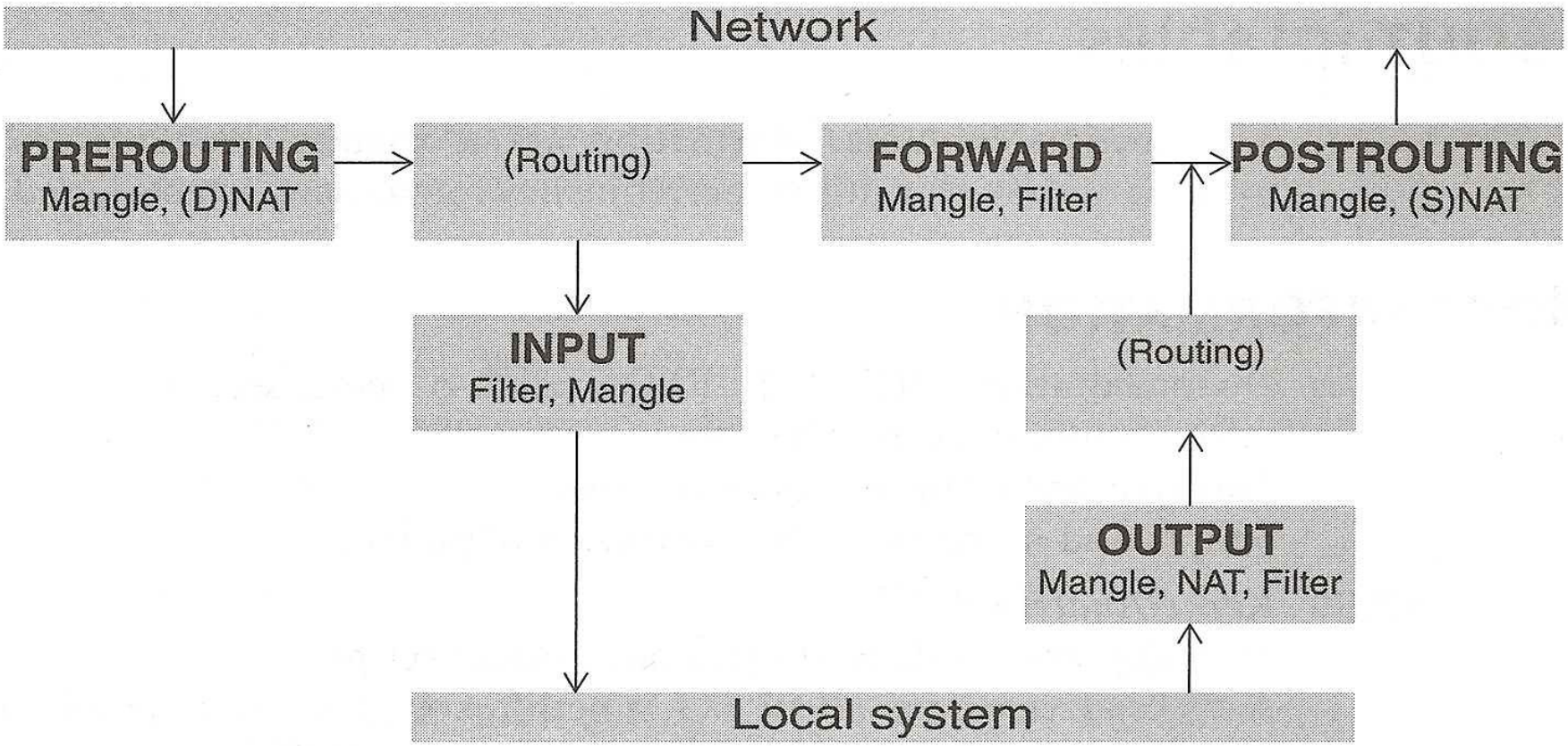
Packet Filtering

- TCP/IP headers contain
  - Source/destination IP address
  - TCP/UDP source/destination port
  - ICMP message type
  - Encapsulated protocol information (TCP, UDP, ICMP or IP tunnel)
- Used to decide what do do with the packet
- Servers should at least block unused ports, in case firewall compromised

Linux Packet Filtering

- Built into kernel – very fast
- Kernel part called netfilter
  - nftables replaces parts after kernel 3.13
- Firewall client software called iptables or firewalld
  - firewalld can interface to iptables or nftables
- Can build very secure firewall using these
  - NT may be too insecure for a serious firewall
- Three tables (netfilter) : Filter, NAT, Mangle

Filtering a Packet in the Kernel



What Is a Rule?

- Each rule specifies conditions the packet must meet and what is done to it if it meets the conditions (called 'target')
  - E.g. drop all ICMP packets coming from 127.0.0.1
  - iptables -A INPUT -s 127.0.0.1 -p icmp -j DROP
- Then to delete this rule:
  - iptables -D INPUT 1
  - iptables -D INPUT -s 127.0.0.1 -p icmp -j DROP
- Consult one of the tutorials or howtos about some reasonable rule sets

Summary: Security

- Note that security is a big subject and firewalls are a small (but important part)
- The iptables utility creates firewalls intended to prevent unauthorized access to a system or network
- Chains of rules are collected in three tables
  - Filter, NAT, Mangle
  - Filter is default table used to DROP or ACCEPT packets based on content

Chapter 28: Programming the Bourne Again Shell

- Continues from previous chapter
- Introduces
  - additional commands
  - builtins
- Takes shell programming to the point where it is useful
- Builds on earlier chapters
  - knowledge of commands is the foundation

Control Structures

- Commands that alter flow of control in a script
- `if ... then`
- `if ... then ... else`
- `if ... then ... elsif`
- `for ... in`
- `for`
- `while and until`
- `case`

if ... then

- Syntax

```
if test-command
then
  commands
fi
```

- Note closing fi
- *test-command* is a command which returns zero (true) or nonzero (false)
  - Note zero is also the success return condition
- The `test` command and `[...]` construct are often used

The `test` Builtin

- `test` *`expresion`* returns true if the expression is true
  - `[`*`expression`*`]` is the same
- Expressions can be combined with && (and), || (or)
- Kinds of test operators
  - unary file operators, e.g. `-e` *`file`* (*file* exists)
  - binary file operators, e.g. *`fileA`* `-nt` *`fileB`* (*fileA* is newer than *fileB*)
  - binary string operators, e.g. *`string1`* `=` *`string2`*
  - binary arithmetic expressions, e.g. `exprA -lt exprB`

File Operators

- `-d` *`file`* (*file* exists and is a directory)
- `-e` *`file`* (*file* exists)
- `-f` *`file`* (*file* exists and is a regular file)
- `-r` *`file`* (*file* exists and is readable)
- `-s` *`file`* (*file* exists and is not empty)
- `-w` *`file`* (*file* exists and is writable)
- `-x` *`file`* (*file* exists and is executable, or is a directory that can be searched)

String Operators

- `string` (true if *string* is non-null)
- `-n string` (*string* is non-null)
- `-z string` (*string* is null, i.e. length of zero)
- `string1 = string2` (*string1* equals *string2*)
- `string1 != string2` (*string1* does not match *string2*)

Arithmetic Expressions

- *exprA* `-eq` *exprB*
- *exprA* `-ne` *exprB*
- *exprA* `-lt` *exprB*
- *exprA* `-gt` *exprB*
- *exprA* `-le` *exprB*
- *exprA* `-ge` *exprB*
- *exprA* `-a` *exprB* (true if *exprA* and *exprB* both true)
  - only in test expression, not in [ ]
- *exprA* `-o` *exprB* (true if *exprA* or *exprB* is true)
  - only in test expresion, not in [ ]
- `!` *exprA* (true if *exprA* is false)

if ... then ... else

- Syntax

```
if test-command
then
    commands
else
    commands
fi
```

- Alternate layout:

```
if test-command; then
    commands
else
    commands
fi
```

if ... then ... elif

- Syntax

```
if test-command
then
    commands
elif test-command
then
    commands
...
else
    commands
fi
```

May have more elif sections

for ... in

- Syntax

```
for loop-index in argument-list
do
    commands
done
```

- Executes the body of the loop with the ***loop-index*** taking on the value of each argument in the ***argument-list*** in turn
- Example:

```
for fruit in apples oranges pears bananas
do
  echo "$fruit"
done
echo "Task complete"
```

for

- The basic for loops through the command-line arguments and assigns each to the loop variable in turn
- Syntax

```
for loop-index
do
    commands
done
```

- Example:

```
for arg
do
    echo "$arg"
done
```

while

- The preferred looping structure when you loop as long as something is true
- Syntax

```
while test-command
do
    commands
done
```

- Example:

```
number=0
while [ "$number" -lt 10 ]
do
    echo -n "$number"
    let number="$number"+1
done
echo
```

Let evaluates arithmetic arguments, including assignment

until

- In the until loop, looping continues until the test condition becomes true
- Syntax

```
until test-condition
do
    commands
done
```

- Example:

```
secretname=caliche
name=""
echo "Try to guess the secret name!"
echo
until [ "$name" = "$secretname" ]
do
   echo -n "Your guess: "
   read name
done
echo "Very good"
```

case

- Selects one of a number of alternatives
- Syntax

```
case test-string in
    pattern-1)
        commands-1
        ;;
    pattern-2)
        commands-2
        ;;
...
esac
```

The Here Document

- A Here document allows redirection of a portion of shell script text to standard input for a comand in the script
- Put `<<somestring` in the script to indicate Here document follows on the next line
- Put `somestring` (without the quotes) by itself on a line to end the document

Summary

- For simple tasks, it is easiest to program them using the shell
- To do this we need to be familiar with the commands and builtins available to us
- You should at this point know the utilities and builtins on the following slides

Useful Utilities

- cat
- cp
- cut
- dd
- diff
- find
- grep
- gzip
- head
- less
- ln
- lpr
- ls
- man
- mkdir
- mv
- od
- rm

More Useful Utilities

- rmdir
- sed
- sort
- tail
- tar
- touch
- uniq
- wc
- ftp
- telnet
- mesg
- pine
- write
- talk
- cd
- chmod
- chown

Still More Useful Utilities

- date
- df
- du
- file
- finger
- kill
- nice
- ps
- sleep
- tr
- stty
- top
- umask
- w
- which
- who
- echo
- tee
- test

Kubernetes Summary: What makes OKD Run?

- Linux mechanisms
  - Kernel namespaces, control groups
- Containers
  - Images, runtime
- Pods
  - Containers with shared localhost
- Kubernetes – organize the pods
  - (Kubernetes namespaces != Linux namespaces)
- OKD – security and convenience features

What Makes a Virtual Machine?

- Conditions defined by Popek and Goldberg in 1974
  - see Wikipedia article for hypervisor (or VM)
- Equivalence/Fidelity
  - programs should exhibit a behavior essentially identical to that when running on an equivalent machine directly
- Resource Control/Safety
  - hypervisor must be in complete control of the virtualized resources
- Efficiency/Performance
  - most machine instructions must be executed directly

Linux Mechanisms

- Linux kernel has mechanisms for creating isolated virtual machine-like containers
  - nftables for networking
  - systemd for starting services
  - namespaces for isolation
  - cgroups (control groups) for metering and limiting resources
- No hardware virtualization required at all!

Containers Run Images

- What do you build from a DockerFile?
  - docker save *imageName* > image.tar
- You can examine tar file and see image layers
  - defined in Open Container Initiative (OCI)
  - most DockerFile commands create new layer
- Each layer overlay previous ones
- Layers can be shared to avoid file transfers
- Build log files show downloading layers

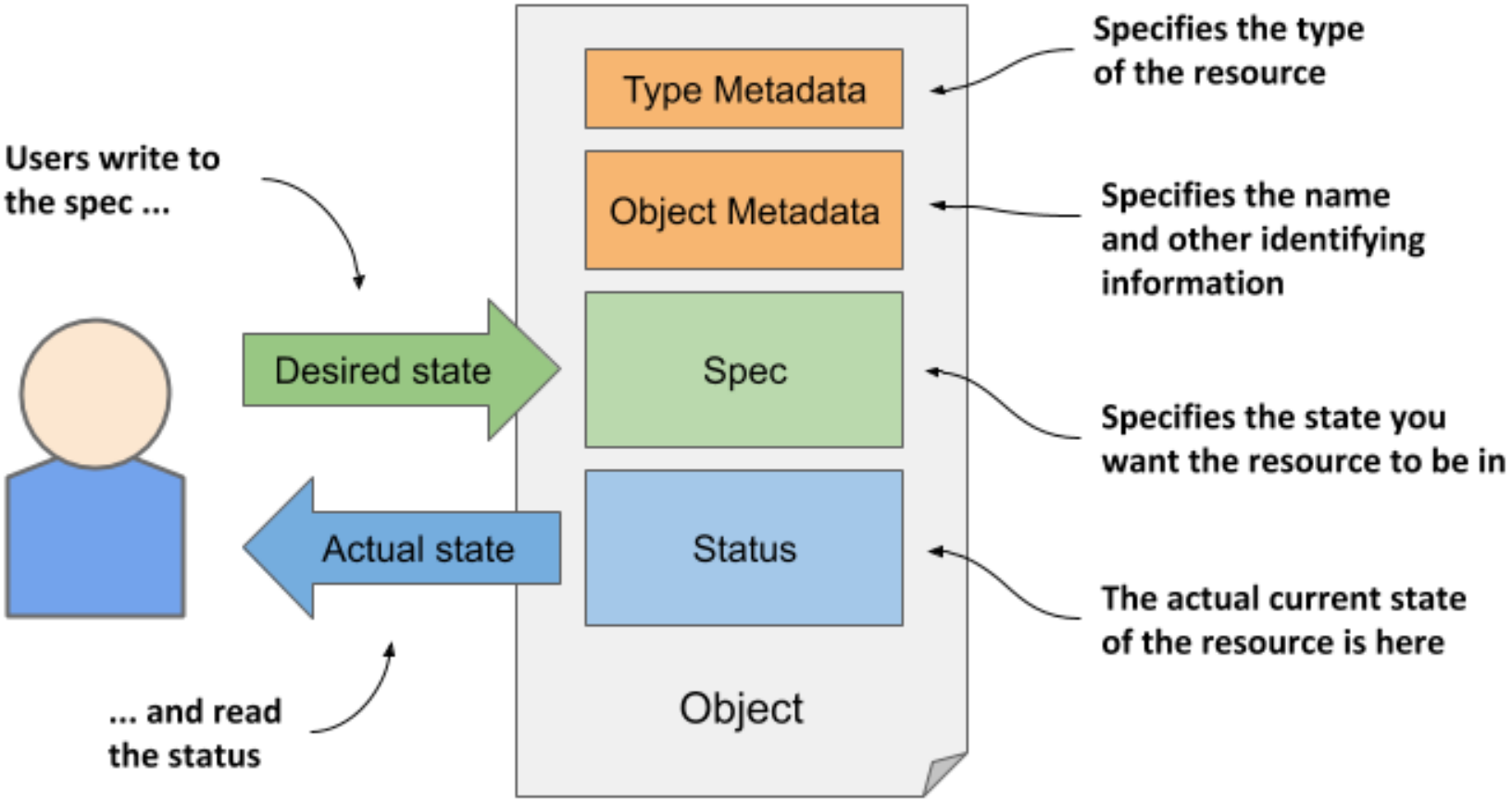
Container Runtimes

- Standard: Container Runtime Interface (CRI)
- See Kubernetes CRI runtimes
- Initially Kubernetes only supported docker
  - can still use docker runtime with docker-shim
  - but *replacing* docker saves 20% of FCOS (~760MB)
  - Kubernetes 1.22 dropping support for docker-shim
- containerd: early alternative to docker
- cri-o: used in OKD (`systemctl status crio`)
- runc: library at base of many container runtimes

Object Manifest

- Type metadata: object type, group and API version
- Object metadata: basic information such as name
- Spec: desired state of object
- Status: current state of object
- Can see using `oc describe` command
  - or web console YAML tabs (e.g. edit deployment)

Kubernetes API Object



From Kubernetes in Action 2nd Edn MEAP, by Marco Luksa, 2021

Custom Resource Definition (CRD)

- Resource: stores collection of API objects of a certain kind
  - E.g. the Pods resource contains Pod objects
- CRD defines new object type and lets API server manage its lifecycle
- Custom Resource (CR) objects created from CRDs
  - added by cluster administrator
- API creates RESTful endpoint to serve CRs

Template for CRDs

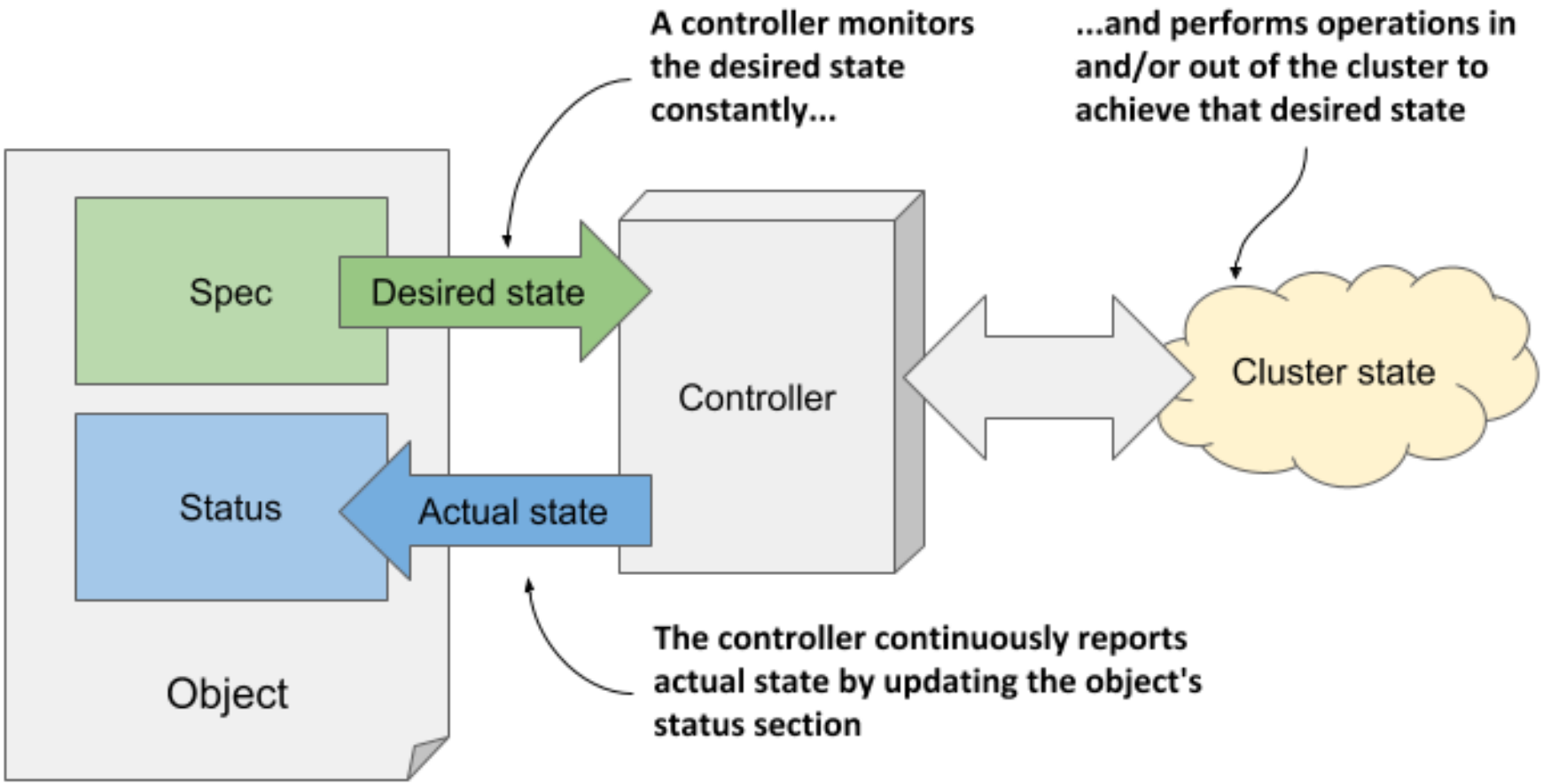
```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: crontabs.stable.example.com
spec:
  group: stable.example.com
  version: v1
  scope: Namespaced
  names:
    plural: crontabs
    singular: crontab
    kind: CronTab
    shortNames:
    - ct
```

See Using CRDs

Controllers

- Kubernetes runs on the controller pattern
  - process requests to change state, make changes, watch for deviation, bring back to desired state
- Controls (for example)
  - binding storage to processes
  - creating and scaling containers
  - terminating and migrating unhealthy containers
  - creating IP routes to ports
  - updating load balanced endpoints

How Controller Manages Object



From Kubernetes in Action 2nd Edn MEAP, by Marco Luksa, 2021

Images

- An image is a binary that includes
  - all requirements for running a single container
  - metadata describing its needs and capabilities
- Based on OCI or Docker formats
- Can be build with podman or Docker
- Can be build with OKD builder images
- Images identified by their hash
- Stored in image registry or repositories

Image Streams

- Abstraction for referencing images in OKD
- Have tags that reference repositories and images
- Builds referring to image streams query the repository to pull image
- Allow specific images to be used even if image in repository changes
- Builds/deployments can watch imagestream for notifications of changed images

Templates (documentation)

- Describe a set of objects that can be parameterized and processed to produce a list of objects for creation by OKD
- Can be used from CLI or web console
- Goes beyond image streams to allow multiple cooperating objects to be created
- Can create from YAML file
- List parameters: `oc process --parameters -n <project> <template name>`

Building Your Own OKD

- Cloud provider or self-hosted
  - AWS, Azure, GCP
  - vmWare, libVirt
- DHCP
  - On firewall, or bastion host
  - pfSense
- DNS
  - bind or dnsmasq

Build OKD

- Layer 4 load balancing
  - HAproxy, F5
- iPXE boot or boot device
  - iPXE can use TFTP to deliver images
- Web server to deliver ignition files
  - Apache, nginx

Build OKD

- Download recent oc
- Use oc to download current oc and openshift-install
- Edit install-config.yaml (next slide)
- Run openshift-install to create ignition files and authentication
- Create bootstrap node, master nodes, worker nodes

Example install-config.yaml Files

```
apiVersion: v1
baseDomain: infoteach.ca
metadata:
  name: okd4

compute:
- hyperthreading: Enabled
  name: worker
  replicas: 0

controlPlane:
  hyperthreading: Enabled
  name: master
  replicas: 3

platform:
  none: {}

fips: false

pullSecret: '{"auths":{"fake":{"auth": "Zm9vOmJhcgo="}}}'
sshKey: 'ssh-rsa AAAAB…..
```

```
apiVersion: v1
baseDomain: %%SNC_DOMAIN%%
metadata:
  name: okd4-snc

networking:
  networkType: OpenShiftSDN
  clusterNetwork:
  - cidr: 10.100.0.0/14
    hostPrefix: 23
  serviceNetwork:
  - 172.30.0.0/16

compute:
- name: worker
  replicas: 0
controlPlane:
  name: master
  replicas: 1
platform:
  none: {}
pullSecret: '{"auths":{"fake":{"auth": "Zm9vOmJhcgo="}}}'
sshKey: %%SSH_KEY%%
```

Summary

- Lots of moving parts here
- You can play with a hobby version of OKD
  - see okd.io blogs
  - OKD 4.18 makes it easier to install single-node cluster in standard way
  - MicroShift is targeting small targets for edge computing
- See okd.io, kubernetes.io for latest activities